

Exercices sur les calculs inductifs

Enseignement à distance de l'Université Marie et Louis Pasteur
Spécification et preuve de programmes

Version du 9 décembre 2025

1 Table de vérité

Sans utiliser aucune fonction prédéfinie sur le type `bool`, définir une fonction WhyML nommée `or` qui implémente le “ou” logique conformément à sa table de vérité.

2 Type inductif des entiers naturels

On adopte la définition inductive suivante du type $\mathbb{N}$ des entiers naturels :

1. 0 est un entier naturel.
2. Si n est un entier naturel, alors $S(n)$ est un entier naturel.
3. L'ensemble $\mathbb{N}$ est le plus petit ensemble qui contient les entiers naturels définis par les règles 1 et 2.

Remarque importante : Pour éviter tout risque de confusion avec l'addition, qu'on définit dans cet exercice, l'entier naturel qui suit immédiatement l'entier naturel n dans l'ordre croissant des entiers n'est pas noté $n + 1$, mais $S(n)$, qui se lit “successeur (immédiat) de n ”.

1. Donner le type d'une fonction *plus* d'arité 2 qui réalise dans $\mathbb{N}$ l'addition de deux entiers naturels.
2. Définir complètement cette fonction *plus* par deux égalités.
3. Définir un type WhyML nommé `nat` pour les entiers naturels, avec les constructeurs `Z` et `S` pour 0 et S , puis coder la fonction *plus* en WhyML, sous la forme d'une fonction récursive nommée `plus`.
4. Utiliser la fonction *plus* pour définir par deux égalités une fonction *mult* qui réalise le produit de deux entiers naturels de $\mathbb{N}$.
5. Donner une traduction de cette définition en WhyML.
6. En copiant ce code WhyML dans l'interface en ligne de Why3, vérifier qu'il est syntaxiquement correct et que le typage est respecté.

3 Fonction miroir

Sans consulter la bibliothèque standard de **Why3**, définir une fonction WhyML qui implémente sur le type des listes défini dans cette bibliothèque standard la fonction *miroir* du cours.

4 Calcul de sommes

1. Donner le type d'une fonction *somme-liste* qui calcule la somme des nombres figurant dans une liste d'entiers relatifs donnée.
2. Décrire cette fonction *somme-liste* par des égalités basées sur la définition inductive du type *liste*(α).
3. En utilisant les types `int` et `list` de WhyML, donner un code WhyML qui définit une fonction `sommeListe` conformément à ces égalités.
4. Donner le type et définir par des égalités une fonction *liste-sommes* qui retourne la liste des sommes des listes d'entiers relatifs figurant dans une liste donnée.
5. Traduire ce type et ces égalités en une définition WhyML d'une fonction `listeSommes`.
6. Donner le type et définir inductivement une fonction *aplatir* qui transforme une liste de listes de valeurs en une liste unique de ces valeurs mises bout à bout, dans le même ordre.
7. Traduire ce type et ces égalités en une définition WhyML d'une fonction `aplatir`.